

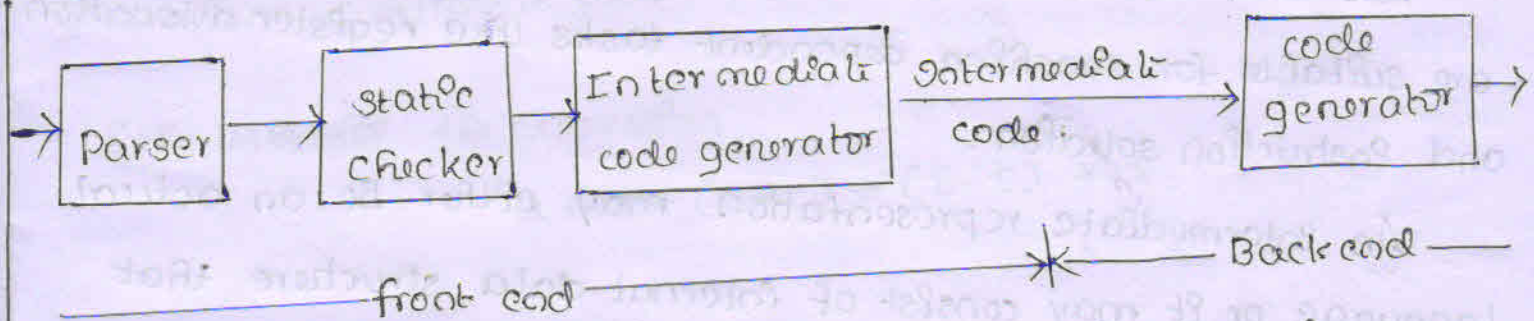
UNIT-6

INTERMEDIATE CODE

GENERATION

Intermediate Code generation.

In the analysis-synthesis model of a compiler, the front end analyzes a source program and creates an intermediate representation, from which the back end generates target code.



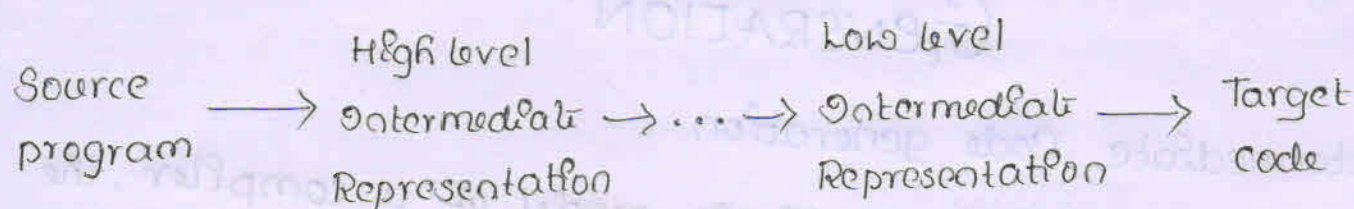
Logical structure of a compiler front end

Parsing, static checking and intermediate code generation are done sequentially; sometimes they can be combined and folded into parsing.

Static checking includes type checking, which ensures that operators are applied to compatible operands. It also includes any syntactic checks that remain after parsing.

Ex: It ensures that a break-statement in C is enclosed within a while-, for- or switch-statement; an error is reported if such an enclosing statement does not exist.

In the process of translating a program in a given source language into code for a given target machine, a compiler may construct a sequence of intermediate representations as



High level representations are close to the source language and are well suited to tasks like static type checking.

Ex: Syntax tree

Low level representations are close to the target machine & are suitable for machine dependent tasks like register allocation and instruction selection.

An intermediate representation may either be an actual language or it may consist of internal data structures that are shared by phases of the compiler.

Variants of Syntax Trees

Nodes in a syntax tree represent constructs in the source program; the children of a node represent the meaningful components of a construct.

A directed acyclic graph (DAG) for an expression identifies the common subexpressions of the expression.

Directed Acyclic Graphs for Expressions.

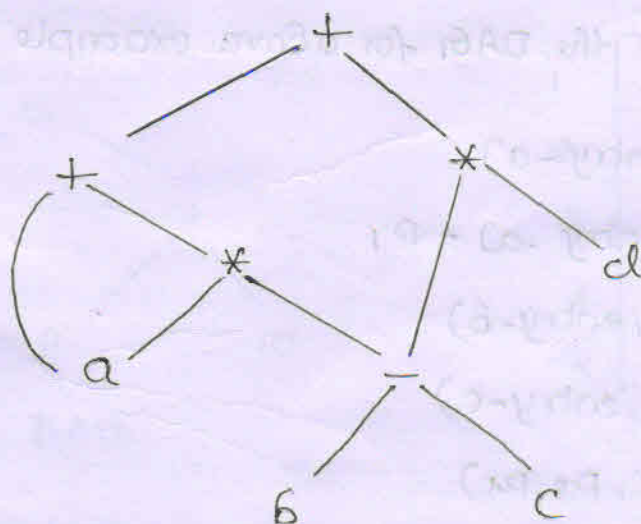
In DAG leaves represents the atomic operands and interior nodes represents the operators. as in the syntax tree.

A node N in a DAG has more than one parent if N represents a common subexpression; But in the syntax tree, the tree for the common subexpression would be duplicated as many times as the subexpression appears in the original expression.

DAG gives the compiler important clues regarding the generation of efficient code to evaluate the expressions.

Ex: DAG for the expression

$$a + a * (b - c) + (b - c) * d$$



↳ The leaf for 'a' has 2 parents, because 'a' appears twice in the expression

↳ The 2 occurrence of the common subexpression $b - c$ are represented by one node, the node labeled '-'

SDD to produce DAG.

PRODUCTION	SEMANTIC RULES
(e) $E \rightarrow E_1 + T$	$E.node = \text{new Node}(' + ', E_1.node, T.node)$
(ee) $E \rightarrow E_1 - T$	$E.node = \text{new Node}(' - ', E_1.node, T.node)$
(eee) $E \rightarrow T$	$E.node = T.node$
(ev) $T \rightarrow (E)$	$T.node = E.node$
(v) $T \rightarrow id$	$T.node = \text{new Leaf}(id, id.entry)$
(ve) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

It will construct a DAG, before creating a new node, these functions first check whether an identical node already exists.

If a previously created identical ~~ex~~ node exists, the existing node is returned.

Steps for constructing the DAG for above example.

(e) $P_1 = \text{Leaf}(id, \text{entry} - a)$

(ee) $P_2 = \text{Leaf}(id, \text{entry} - a) = P_1$

(eee) $P_3 = \text{Leaf}(id, \text{entry} - b)$

(ev) $P_4 = \text{Leaf}(id, \text{entry} - c)$

(v) $P_5 = \text{Node}(' - ', P_3, P_4)$

(ve) $P_6 = \text{Node}(' x ', P_1, P_5)$

(vee) $P_7 = \text{Node}(' + ', P_1, P_6)$

(veee) $P_8 = \text{Leaf}(id, \text{entry} - b) = P_3$

(ex) $P_9 = \text{Leaf}(id, \text{entry} - c) = P_4$

(x) $P_{10} = \text{Node}(' - ', P_3, P_4) = P_5$

(xe) $P_{11} = \text{Leaf}(id, \text{entry} - d)$

Ex: $P_{12} = \text{Node}('x', P_5, P_{11})$

Ex: $P_{13} = \text{Node}('+', P_7, P_{12})$

When the call to Leaf (Ed, entry- a) is repeated at step 2, the node created by the previous call is returned, so $P_2 = P_1$.

The Value-Number Method for Constructing DAG's

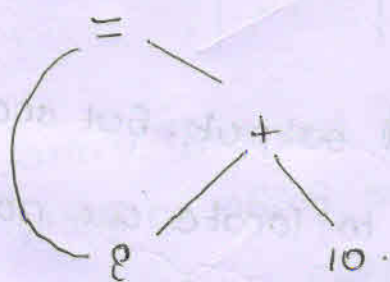
* The nodes of a DAG are stored in an array of records

* Each row of array represents one record & therefore one

node.

* In each record, the first field is an operation code, indicating the label of the node. Leaves have one additional field which holds the lexical value and interior nodes have 2 additional fields indicating the left and right children.

Ex: DAG for $e = e + 10$ allocated in an Array



DAG

1	Ed		
2	num	10	
3	+	1	2
4	=	1	3
5			

to entry
for e

Array

* In this array, we refer to nodes by giving the integer index of the record for that node within the array.

* This integer historically has been called the "value number" for the node or for the expression represented by the node

* For above example node labeled + has value number 3 & its left & right children have value numbers 1 & 2 respectively.

Suppose that nodes are stored in an array & each node is referred to by its value number. Let the signature of an interior node be the triple $\langle op, l, r \rangle$ where op is the label, l is its left child's value number & r its right child's value number. A unary operator may be assumed to have $r=0$.

ALGORITHM: To construct the nodes of a DAG using value number method.

INPUT: Label op , node l and node r

OUTPUT: The value number of a node in the array with signature $\langle op, l, r \rangle$

METHOD: Search the array for a node M with label op , left child l and right child r . If there is such a node, return the value number of M . If not, create in the array a new node N with label op , left child l and right child r & return its value number.

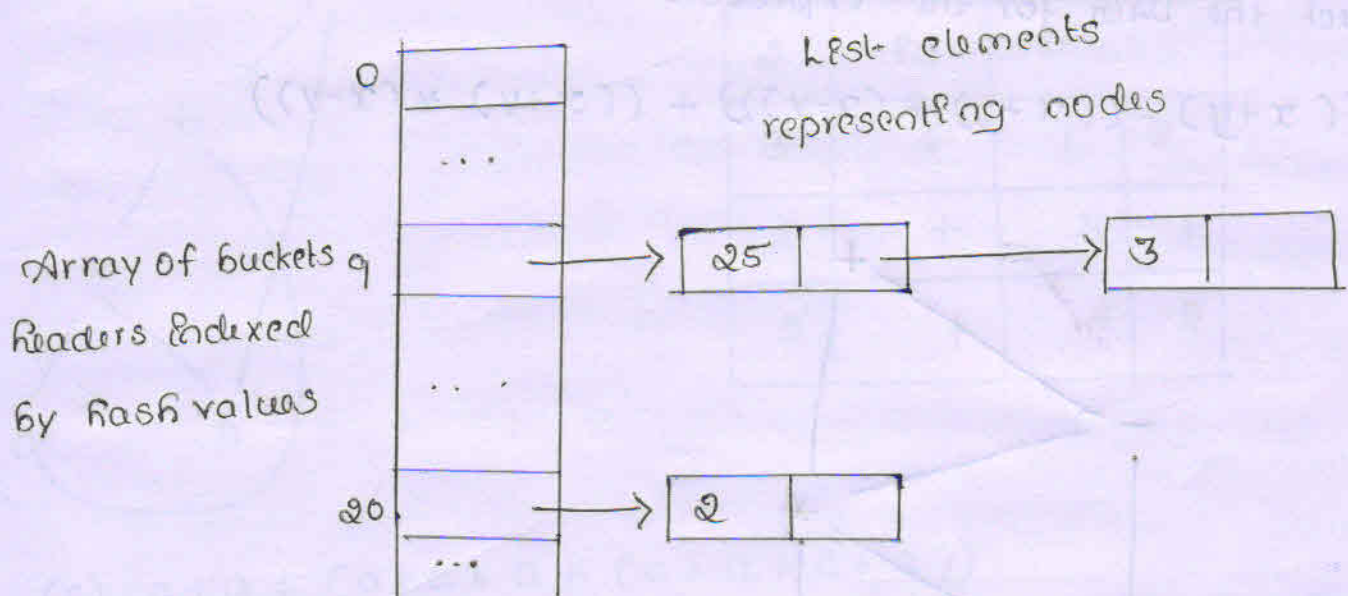
Above algorithm yields the desired output, but searching the entire array every time we are asked to locate one node is expensive.

A more efficient approach is to use a hash table, in which the nodes are put into "buckets" each of which typically will have only a few nodes. It supports dictionaries, which is an abstract data type that allows us to insert & delete elements of a set & to determine whether a given element is currently in the set.

To construct a hash table for the nodes of a DAG, we need a hash function R that computes the index of the bucket for a signature $\langle op, l, r \rangle$.

The bucket index $R(\langle op, l, r \rangle)$ is computed deterministically from op, l & r so that we may repeat the calculation & always get to the same bucket index for node $\langle op, l, r \rangle$.

The buckets can be implemented as linked list as,



An array indexed by hash value, holds the bucket pointers, each of which points to the first cell of a list. Within the linked list for a bucket, each cell holds the value number of one of the nodes that hash to that bucket. That is, node $\langle op, l, r \rangle$ can be found on the list whose reader is at index $R(\langle op, l, r \rangle)$ of the array.

Thus, given the output input node op, l & r we compute the bucket index $R(\langle op, l, r \rangle)$ & search the list of cells in this bucket for the given input node.

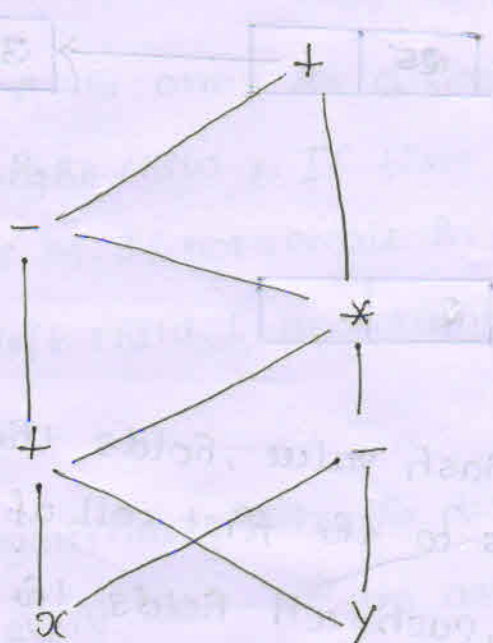
For each value number 'v' found in a cell, we must

check whether the signature $\langle op, l, r \rangle$ of the input node matches the node with value number v in the list of the cell. If we find a match, we return v . If we find no match, we know no such node can exist in any other bucket, so we create a new cell, add it to the list of cells for bucket-index $R(op, l, r)$ & return the value number in that new cell.

Problems.

Construct the DAG for the expression.

$$((x+y) - ((x+y) * (x-y))) + ((x+y) * (x-y))$$



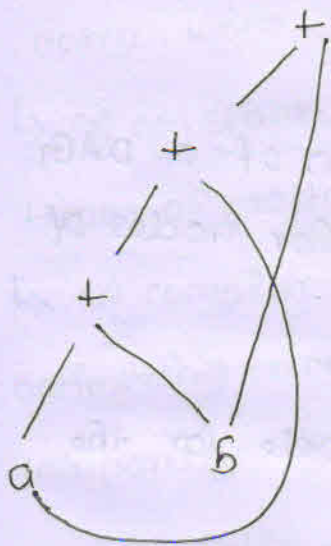
Construct the DAG & identify the value number for the sub expressions of the following expressions, assuming + associated from the left.

(a) $a + 6 + (a + 6)$



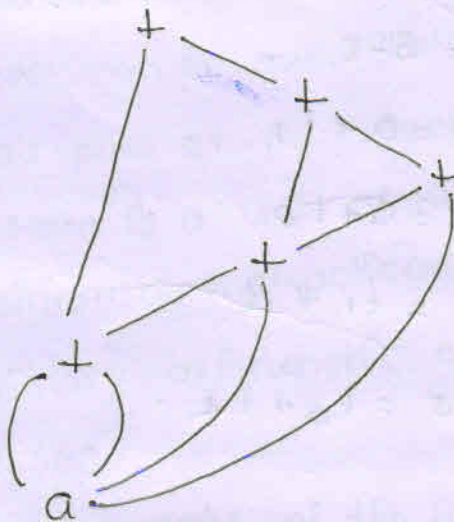
1	ld	a	
2	ld	6	
3	+	1	2
4	+	3	3

(6) $a + 6 + a + 6$



1	ld	a	
2	ld	6	
3	+	1	2
4	+	3	1
5	+	4	2

(c) $a + a + (a + a + a + (a + a + a + a))$



1	ld	a	
2	+	1	1
3	+	2	1
4	+	3	1
5	+	3	4
6	+	2	5

Three-Address Code

In 3-address code, there is at most one operator on the right side of an instruction, i.e., no built up arithmetic expression are permitted.

Thus a source-language expression like $x + y * z$ might be translated into the sequence of 3 address instructions

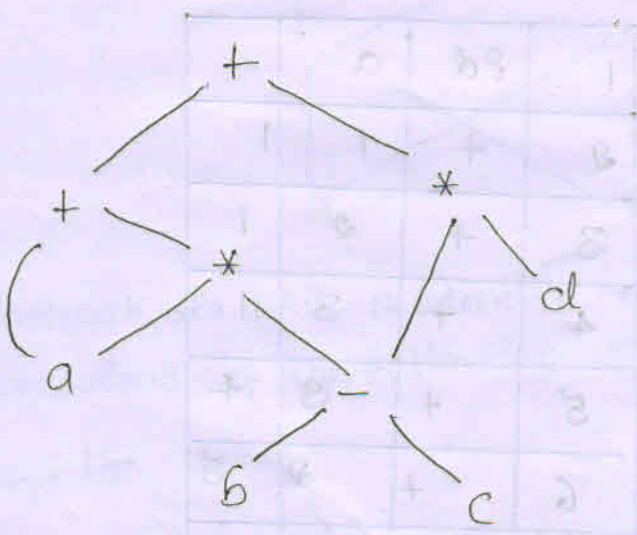
$$t_1 = y * z$$

$$t_2 = x + t_1$$

where t_1 & t_2 are compiler generated names.

3 address code is a linearized representation of a DAG in which explicit names correspond to the interior nodes of the graph.

Ex: Write DAG & its corresponding 3 address code for the expression $a + a * (b - c) + (b - c) * d$.



DAG

$$t_1 = b - c$$

$$t_2 = a * t_1$$

$$t_3 = a + t_2$$

$$t_4 = t_1 * d$$

$$t_5 = t_3 + t_4$$

3-address code

Addresses and Instructions.

3-address code is built from 2 concepts: address & instructions.

An address can be one of the following.

- ↳ A name: For convenience, we allow source program names to appear as addresses in 3-address code. In an implementation, a source name is replaced by a pointer to its symbol table entry, where all information about the name is kept.
- ↳ A constant: A compiler must deal with many different types of constants and variables.
- ↳ A compiler generated temporary: It is useful, especially in optimizing compilers, to create a distinct name each time a temporary is needed.

Symbolic labels will be used by instructions that alter the flow of control. A symbolic label represents the index of a 3-address instruction in the sequence of instructions. Actual indexes can be substituted for the labels, either by making a separate pass or by "backpatching".

Here is a list of the common 3-address instruction forms

(E) Assignment instructions of the form $x = y \text{ op } z$ where op is a binary arithmetic or logical operation & x, y & z are addresses.

(EE) Assignments of the form $x = \text{op } y$, where op is a unary operation

(EEE) Copy instructions of the form $x = y$, where x is assigned the value of y .

(iv) An unconditional jump goto L. The 3-address instruction with label L is the next to be executed.

(v) Conditional jumps of the form if x goto L and if False x goto L. These instructions execute the instruction with label L next if x is true and false respectively.

(vi) Conditional jumps such as if x relop y goto L, which apply a relational operator ($<$, $=$, $>$ etc) to x & y & execute the instruction with label L next if x stands in relation relop to y. If not, the 3 address instruction following if x relop y goto L is executed next, in sequence.

(vii) Procedures calls & returns are implemented using the following instructions: param x for parameters; call p, n & y = call p, n for procedure & function calls respectively & return y, where y representing a returned value, is optional

param x_1

param x_2

...

param x_n

call p, n

The integer n indicating the no. of actual parameters in call p, n is not redundant because calls can be nested.

(viii) Indexed copy instructions of the form $x = y[p]$ and $x[p] = y$. The instruction $x = y[p]$ sets x to the value in the location p memory units beyond location y. The instruction $x[p] = y$ sets the contents of the location p units beyond x to the value of y.

(ix) Address & pointer assignments of the form $x = \&y$, $x = *y$ and $*x = y$.

Ex: Consider the statement

do $i = i + 1$; while ($a[i] < v$);

2 possible translations of this statement are

L: $t_1 = i + 1$

$i = t_1$

$t_2 = i * 8$

$t_3 = a[t_2]$

if $t_3 < v$ goto L.

100: $t_1 = i + 1$

101: $i = t_1$

102: $t_2 = i * 8$

103: $t_3 = a[t_2]$

104: if $t_3 < v$ goto 100

(6) position number.

(a) symbolic labels

The translation in (a) uses a symbolic label L, attached to the first instruction. The translation in (b) shows position number for the instructions, starting arbitrarily at position 100. In both translations, the last instruction is a conditional jump to the first instruction. The multiplication $i * 8$ is appropriate for an array of elements that each take 8 units of space.

Quadruples

A quadruple has 4 fields, which we call op, arg1, arg2, & result. The op field contains an internal code for the operator.

Ex: $x = y + z$ is represented by placing + in op, y in arg1, z in arg2 & x in result.

The following are some exceptions to this rule.

(e) Instructions with unary operators like $x = \text{minus } y$ or $x = y$ do not use arg_2 . Note that for a copy statement like $x = y$, op is $=$, while for most other operations, the assignment operator is implied.

(ee) Operators like param use neither arg_2 nor result .

(eee) Conditional & unconditional jumps put the target label in result .

Ex: Write quadruples for $a = 6 * -c + 6 * -c$.

$t_1 = \text{minus } c$

$t_2 = 6 * t_1$

$t_3 = \text{minus } c$

$t_4 = 6 * t_3$

$t_5 = t_2 + t_4$

$a = t_5$

	op	arg1	arg2	result
0	minus	c		t_1
1	*	6	t_1	t_2
2	minus	c		t_3
3	*	6	t_3	t_4
4	+	t_2	t_4	t_5
5	=	t_5		a

(a) 3-address code

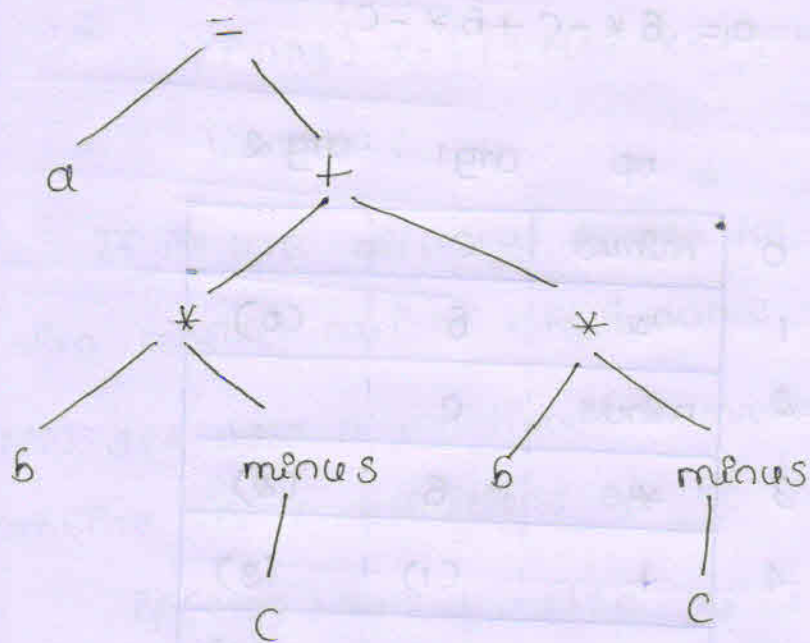
(b) Quadruples.

The special operator minus is used to distinguish the unary minus operator.

Triples

A triple has only 3 fields, op , arg_1 & arg_2 . Note that the result field in quadruples is used primarily for temporary names. Using triples, we refer to the result of the operation $x \text{ op } y$ by its position, rather than by an explicit temporary name.

Ex: write triples for $a = 6 * -c + 6 * -c$



(a) Syntax tree

	op	arg 1	arg 2
0	minus	c	
1	*	6	(0)
2	minus	c	
3	*	6	(2)
4	+	(1)	(3)
5	=	a	(4)

(6) triples.

A ternary operator like $x[y] = y$ requires 2 entries in the triple structure, for example, we can put x & y in one triple & y in the next.

A benefit of quadruples over triples can be seen in an optimizing compiler, where instructions are often moved around. With quadruples, if we move an instruction that computes a temporary t , then the instructions that use t require no change. With triples, the result of an operation is referred to by its position, so moving an instruction may require us to change all references to that result.

Indirect triples.

Indirect triples consist of a listing of pointers to triples, rather than a listing of triples themselves.

With indirect triples, an optimizing compiler can move an

instruction by reordering the instruction list without affecting the triples themselves.

Ex: Write indirect triples for $a = 6 * -c + 6 * -c$.

Instruction		op	arg1	arg2
35	(0)	minus	c	
36	(1)	*	6	(0)
37	(2)	minus	c	
38	(3)	*	6	(2)
39	(4)	+	(1)	(3)
40	(5)	=	a	(4)
	...			

Static Single - Assignment Form

Static single assignment form (SSA) is an intermediate representation that facilitates certain code optimization.

2 distinctive aspects of SSA that distinguish SSA from 3-address code

(e) All assignments in SSA are to variables with distinct names.

Ex: $p = a + b$
 $q = p - c$
 $p = q * d$
 $p = e - p$
 $q = p + q$

3-address code

$p_1 = a + b$
 $q_1 = p_1 - c$
 $p_2 = q_1 * d$
 $p_3 = e - p_2$
 $q_2 = p_3 + q_1$

Static single assignment form

The same variable may be defined in 2 different control flow paths in a program. For example, the source program

```
if (flag) x = -1; else x = 1;
```

```
y = x * a;
```

If we use different names for x in the true part & false then conflict arises which name should use in $y = x * a$.
(SSA) SSA uses a notational convention called ϕ -function to combine the 2 definitions of x

```
if (flag)  $x_1 = -1$ ; else  $x_2 = 1$ ;
```

```
 $x_3 = \phi(x_1, x_2)$ ;
```

Here $\phi(x_1, x_2)$ has the value x_1 if the control flow passes through the true part of the conditional & the value x_2 if the control flow passes through the false part.

Translate the arithmetic expression $a + -(b + c)$ into

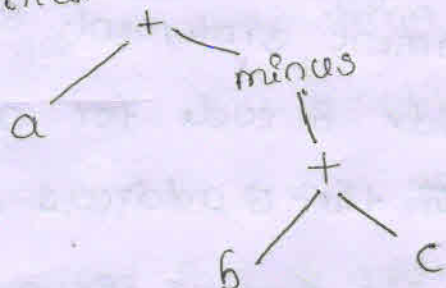
(a) A syntax tree

(b) Quadruples

(c) Triples

(d) Indirect triples

(a) Syntax tree



$$t_1 = b + c$$

$$t_2 = \text{minus } t_1$$

$$t_3 = a + t_2$$

(6) Quadruples

	op	arg 1	arg 2	result
0	+	b	c	t ₁
1	minus	t ₁		t ₂
2	=	a	t ₂	t ₃

(c) triples

	op	arg 1	arg 2
0	+	b	c
1	minus	(0)	
2	=	a	(1)

(d) Indirect triples.

Instructions

35	(0)
36	(1)
37	(2)

	op	arg 1	arg 2
0	+	b	c
1	minus	(0)	
2	=	a	(1)

Translation of Expressions.

An expression with more than one operator, like $a + b * c$, will translate into instructions with at most one operator per instruction. An array reference $A[i][j]$ will expand into a sequence of 3-address instructions that calculate an address for the reference.

Operations with expressions

The following syntax-directed definition builds up the 3-address code for an assignment statement S using attribute code for S & attribute `addr` & code for an expression E . Attributes $S.code$ & $E.code$ denotes the 3 address code for S & E respectively. Attribute $E.addr$ denotes the address

that will hold the value of E .

PRODUCTION	SEMANTIC RULES
$S \rightarrow Ed = E;$	$S.code = E.code \parallel$ $gen(top, get(Ed.lexeme) '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = new temp()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr '=' E_1.addr '+' E_2.addr)$
$l - E_1$	$E.addr = new temp()$ $E.code = E_1.code \parallel$ $gen(E.addr '=' 'minus' E_1.addr)$
$l(E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
lEd	$E.addr = top.get(Ed.lexeme)$ $E.code = ''$

Consider $E \rightarrow Ed$, when an expression is a single identifier say x then x itself holds the value of the expression. In semantic rules $E.addr$ point to the symbol table entry. If top denote the current symbol table then function $top.get$ retrieves the entry when it is applied to the string representation $Ed.lexeme$. $E.code$ is set to empty string.

For $E \rightarrow E_1 + E_2$, compute value of E from E_1 & E_2 . Values are computed into newly generated temporary names. If E_1 is computed into $E_1.addr$ & E_2 into $E_2.addr$ then $E_1 + E_2$

translates into $t = e_1.addr + e_2.addr$, where t is a new temporary name. $e.addr$ is set to t .

The notation $gen(x = 'y' + 'z')$ is used to represent the 3 address instruction $x = y + z$. Variables like x, y & z are evaluated & when passed to gen & quoted strings like $' '$ are taken literally.

$e.code$ is build up by concatenating $e_1.code$, $e_2.code$ & an instruction that adds the values of e_1 & e_2 . The instruction puts the result of the addition into a new temporary name for e , denoted by $e.addr$.

The production $g \rightarrow ld = e$; generates instructions that assign the value of expression e to the identifier ld . The semantic rule for this production uses function $top.get$ to determine the address of the identifier represented by ld .

$g.code$ consists of the instructions to compute the value of e into an address given by $e.addr$, followed by an assign to the address $top.get(ld.lexeme)$ for this instance of ld .

Incremental Translation.

In incremental translation, instead of building up $e.code$ we can arrange to generate only the new 3-address instruction.

In the incremental approach, gen not only constructs a 3 address instruction, it appends the instruction to the sequence of instruction generated so far.

With the incremental approach, the code attribute is not used, since there is a single sequence of instructions that is created by successive calls to gen .

PRODUCTION	SEMANTIC RULES
$S \rightarrow Id = E$	$\{ \text{gen}(\text{top.get}(Id.\text{lexeme} = E.\text{addr}); \}$
$E \rightarrow E_1 + E_2$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} = E_1.\text{addr} + E_2.\text{addr}); \}$
$1 \rightarrow E_1$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} = \text{'minus'} E_1.\text{addr}); \}$
$1 \mid (E_1)$	$\{ E.\text{addr} = E_1.\text{addr}; \}$
$1 \mid Id$	$\{ E.\text{addr} = \text{top.get}(Id.\text{lexeme}); \}$

The semantic rule for $E \rightarrow E_1 + E_2$ simply calls gen to generate an add instruction the instructions to compute E_1 into $E_1.\text{addr}$ & E_2 into $E_2.\text{addr}$ have already been generated.

The new semantic action for $E \rightarrow E_1 + E_2$ creates a node by using a constructor, as in

$E \rightarrow E_1 + E_2 \quad \{ E.\text{addr} = \text{new Node}('+', E_1.\text{addr}, E_2.\text{addr}); \}$

Here, attribute addr represents the address of a node rather than a variable or constant.

Addressing array elements.

Array elements can be accessed quickly if they are stored in a block of consecutive locations. If the width of each array element is w , then the i^{th} element of array A begins in location

$$\text{base} + i \times w \quad \text{--- (1)}$$

Base is the relative address of $A[0]$

The formula (1) generalizes to 2 or more dimensions. In 2 dimension, $A[p_1][p_2]$, let w_1 be the width of a row & let w_2 be the width of an element in a row. The relative address of $A[p_1][p_2]$ can then be calculated by

$$\text{Base} + p_1 * w_1 + p_2 * w_2 \quad - (2)$$

In k dimensions, the formula is,

$$\text{Base} + p_1 * w_1 + p_2 * w_2 + \dots + p_k * w_k \quad - (3)$$

The relative address can be calculated in terms of no. of elements n_j along dimension j of the array and the width $w = w_k$ of a single element of the array.

In 2 dimensions, the location of $A[p_1][p_2]$ is given by

$$\text{Base} + (p_1 * n_2 + p_2) * w \quad - (4)$$

In k dimensions,

$$\text{Base} + ((\dots ((p_1 * n_2 + p_2) * n_3 + p_3) \dots) * n_k + p_k) * w \quad - (5)$$

The array elements need not be numbered starting at 0.

In a one-dimensional array, the array elements are numbered $\text{low}, \text{low}+1, \dots, \text{high}$ & Base is the relative address. (1) can be replaced as

$$\text{Base} + (p - \text{low}) * w \quad - (6)$$

Both (1) & (6) can be rewritten as

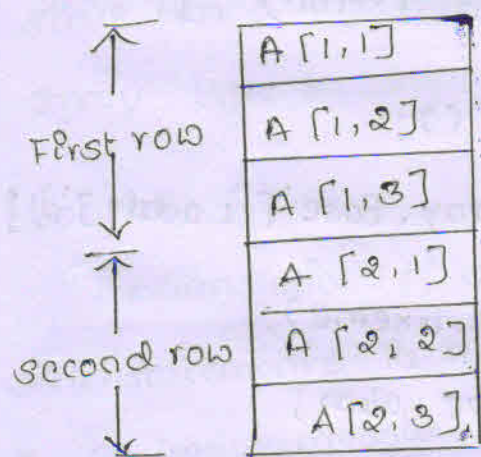
$$p * w + C \quad \text{where,}$$

$$C = \text{Base} - \text{low} * w$$

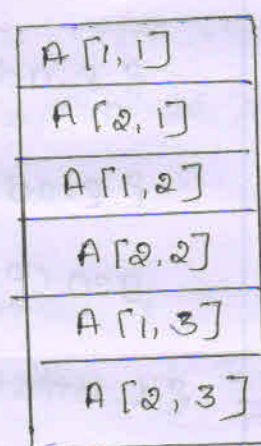
$$C = \text{Base} - \text{low} * w \quad \text{calculated at compile time}$$

Compile time precalculation can not be used when the array's size is dynamic. If we don't know the values of low & high at compile time, then we can't compute constants such as c .

The above address calculations are based on row-major layout for arrays. A 2-dimensional array is normally stored in either row-major or column major forms.



(a) Row major



(b) Column major

We can generalize row or column major form to many dimensions. The generalization of row-major form is to store the elements in such a way that, as we scan down a block of storage, the rightmost subscripts appear to vary fastest, like the numbers on an odometer. Column-major form generalizes to the opposite arrangement, with the leftmost subscripts varying fastest.

Translation of Array References.

Let nonterminal L generate an array name followed by a sequence of index expressions

$$L \rightarrow L[e] \mid id[e]$$

PRODUCTION	SEMANTIC RULES
$S \rightarrow Ed = E;$	$\{ \text{gen}(\text{top.get}(Ed.\text{lexeme}) \text{'=' } E.\text{addr}); \}$
$ L = E;$	$\{ \text{gen}(L.\text{array}.\text{base} \text{'[' } L.\text{addr} \text{']'} \text{'=' } E.\text{addr}); \}$
$E \rightarrow E_1 + E_2$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} \text{'=' } E_1.\text{addr} \text{'+' } E_2.\text{addr}); \}$
$ Ed$	$\{ E.\text{addr} = \text{top.get}(Ed.\text{lexeme}); \}$
$ L$	$\{ E.\text{addr} = \text{new Temp}();$ $\text{gen}(E.\text{addr} \text{'=' } L.\text{array}.\text{base} \text{'[' } L.\text{addr} \text{']'}); \}$
$L \rightarrow Ed [E]$	$\{ L.\text{array} = \text{top.get}(Ed.\text{lexeme});$ $L.\text{type} = L.\text{array}.\text{type}.\text{elem};$ $L.\text{addr} = \text{new Temp}();$ $\text{gen}(L.\text{addr} \text{'=' } E.\text{addr} \text{'*' } L.\text{type}.\text{width}); \}$
$ L_1 [E]$	$\{ L.\text{array} = L_1.\text{array};$ $L.\text{type} = L_1.\text{type}.\text{elem};$ $t = \text{new Temp}();$ $L.\text{addr} = \text{new Temp}();$ $\text{gen}(t \text{'=' } E.\text{addr} \text{'*' } L.\text{type}.\text{width});$ $\text{gen}(L.\text{addr} \text{'=' } L_1.\text{addr} \text{'+' } t); \}$

~~Notes~~ Nonterminal L has 3 synthesized attributes
 (b) $L.\text{addr}$ denotes a temporary that is used while computing
 the offset for the array references by summing the terms
 $\{ l_1 \times w_1 \}$

(EE) L.array is a pointer to the symbol table entry name. The base address of the array, say, L.array.base is used to determine the actual L-value of an array reference - after all the index expressions are analyzed.

(EEE) L.type is the type of the subarray generated by L. For any type t , we assume that its width is given by $t.width$. We use types as attributes, rather than widths, since types are needed anyway for type checking. For any array type t , suppose that $t.elem$ gives the element type.

Control Flow

The translation of statements such as if-else-statements & while-statements is tied to the translation of boolean expression. The boolean expression are often used to

(E) Alter the flow of control

(EE) Compute logical values

Boolean Expressions.

Boolean expressions are composed of the boolean operators applied to elements that are boolean variables or relational expression.

Relational expression are of the form $E_1 \text{ rel } E_2$ where E_1 & E_2 are arithmetic expression & rel.op is used to indicate which of the 6 comparison operators $<, <=, =, !=, >$ or $>=$ is represented by rel .

The semantic definition of the programming language determines whether all part of a boolean expression must be evaluated.

Ex: On expression $B_1 \parallel B_2$, If B_1 is true then the expression is true.

On expression $B_1 \&\& B_2$ If B_1 is false then the entire expression is false.

Short-Circuit code

In short circuit (or jumping) code, the boolean operators $\&\&$, $\parallel$ & $!$ translate into jumps. The operators themselves do not appear in the code, instead, the value of a boolean expression is represented by a position in the code sequence.

Ex: If $(x < 100 \parallel x > 200 \&\& x \neq y) \ x = 0;$

This statement might be translated into the code

if $x < 100$ goto L_2

If false $x > 200$ goto L_1

If false $x \neq y$ goto L_1

$L_2 : x = 0$

$L_1 :$

The boolean expression is true if control reaches label

Flow-of-Control Statements

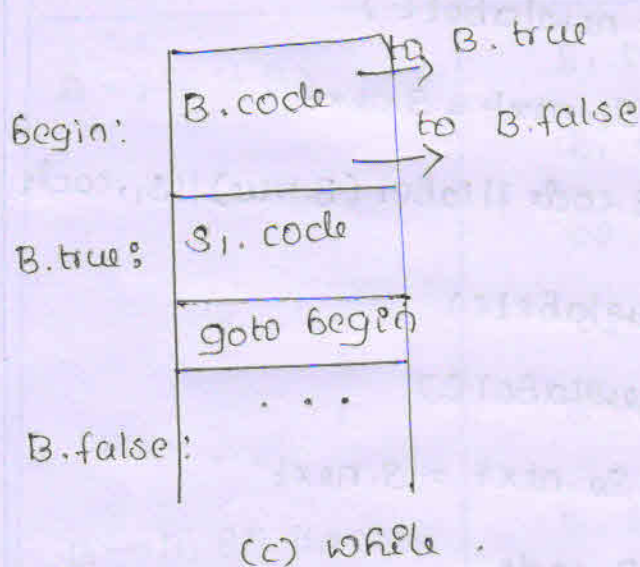
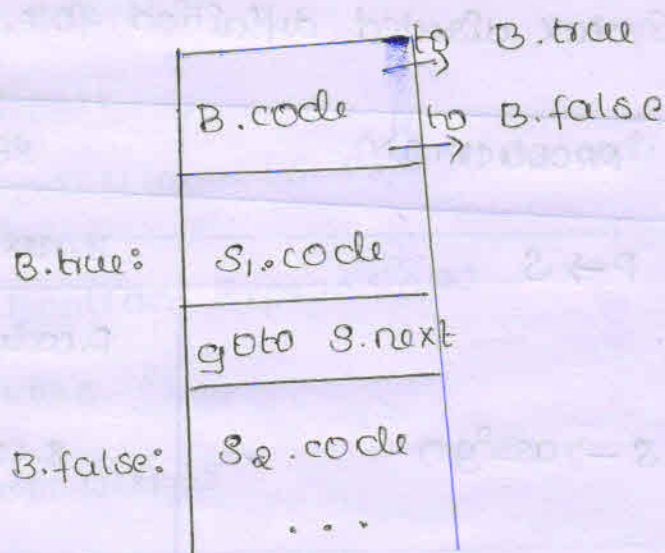
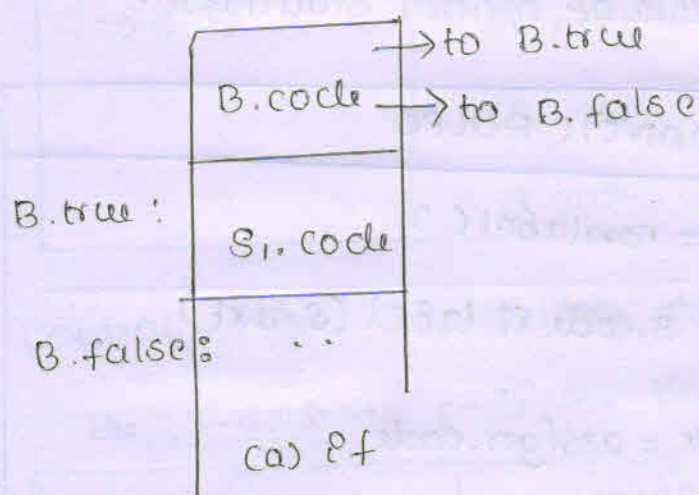
Consider the grammar

$S \rightarrow \text{if } (B) \ S_1$

$S \rightarrow \text{if } (B) \ S_1 \text{ else } S_2$

$S \rightarrow \text{while } (B) \ S_1$

In these productions, nonterminal B represents a boolean expression & non terminal S represents a statement.



The translation of $\text{if } (B) S_1$ consists of B.code followed by S_1 .code. Within B.code are jumps based on the value of B. If B is true, control flows to the first instruction of S_1 .code & if B is false, control flows to the instruction immediately following S_1 .code

With boolean expression B, we associate 2 labels

(e) B.true, the label to which control flows if B is true

(ee) B.false, the label to which control flows if B is false

With statement S, we associate an inherited attribute

$S.next$ denoting a label for the instruction immediately after the code for S

Syntax directed definition for flow of control statement.

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow assign$	$S.code = assign.code$
$S \rightarrow if(B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow if(B) S_1 else S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow while(B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$

$S \rightarrow S_1 S_2$	$\parallel \text{label}(B.\text{true}) \parallel S_1.\text{code}$ $\parallel \text{gen}('goto' \text{ begin})$ $S_1.\text{next} = \text{newlabel}()$ $S_2.\text{next} = S.\text{next}$ $S.\text{code} = S_1.\text{code} \parallel \text{label}(S_1.\text{next}) \parallel S_2.\text{code}$
-------------------------	--

Control - Flow Translation of Boolean Expressions

The 3-address code for booleans is,

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \parallel B_2$	$B_1.\text{true} = B.\text{true}$ $B_1.\text{false} = \text{newlabel}()$ $B_2.\text{true} = B.\text{true}$ $B_2.\text{false} = B.\text{false}$ $B.\text{code} = B_1.\text{code} \parallel \text{label}(B_1.\text{false}) \parallel B_2.\text{code}$
$B \rightarrow B_1 \&\& B_2$	$B_1.\text{true} = \text{newlabel}()$ $B_1.\text{false} = B.\text{false}$ $B_2.\text{true} = B.\text{true}$ $B_2.\text{false} = B.\text{false}$ $B.\text{code} = B_1.\text{code} \parallel \text{label}(B_1.\text{true}) \parallel B_2.\text{code}$
$B \rightarrow ! B_1$	$B_1.\text{true} = B.\text{false}$ $B_1.\text{false} = B.\text{true}$ $B.\text{code} = B_1.\text{code}$
$B \rightarrow E_1 \text{ rel } E_2$	$B.\text{code} = E_1.\text{code} \parallel E_2.\text{code}$

$B \rightarrow \text{true}$	$\text{ gen('if' } \epsilon_1.\text{addr, and, op } \epsilon_2.\text{addr 'goto' } B.\text{true})$
$B \rightarrow \text{false}$	$\text{ gen('goto' } B.\text{false})$ $B.\text{code} = \text{gen('goto' } B.\text{true})$ $B.\text{code} = \text{gen('goto' } B.\text{false})$

$B \rightarrow \epsilon_1 \text{ and } \epsilon_2$ is translated directly into a comparison 3-address instruction with jumps to the appropriate place.

Ex: of B of the form $a < b$

if $a < b$ goto $B.\text{true}$

goto $B.\text{false}$

The remaining production for B translated as
 (i) Suppose B is $B_1 \text{ || } B_2$. If B_1 is true, then B itself is true, $B_1.\text{true}$ is same as $B.\text{true}$. If B_1 is false then B_2 must be evaluated, make $B_1.\text{false}$ be the label of the first instruction in the code for B_2 .

(ii) The translation of $B_1 \&\& B_2$ is similar.

(iii) No code is needed for an expression B of the form $!B_1$; just interchange the true & false exits of B to get the true & false exits of B .

(iv) The constants true & false translate into jumps to $B.\text{true}$ & $B.\text{false}$ respectively.

Ex:

consider if $(x < 100 \text{ || } x > 200 \&\& x \neq y) \text{ } x = 0;$

if $x < 100$ goto L_2

goto L_3

L_3 : if $x > 200$ goto L_4

↳ goto L_1

L_4 : if $x \neq y$ goto L_2

goto L_1

L_2 : $x = 0$

L_1 :

control flow translation of a simple-if-statement

Avoiding Redundant Gotos

of the code is,

if $x > 200$ goto L_4

goto L_1

L_4 :

Instead, consider the instruction:

ifFalse $x > 200$ goto L_1

L_4 :

This ifFalse instruction takes advantage of natural flow from one instruction to the next in sequence, so control simply "falls through" to label L_4 if $x > 200$, thereby avoiding jump. In if-while statements we can use special label fall. The new rules for $S \rightarrow \text{if}(B) S_1$ set $B.\text{true}$ to fall

$B.\text{true} = \text{fall}$

$B.\text{false} = S_1.\text{next} = S_1.\text{next}$

$$S.code = B.code \parallel S_1.code$$

The semantic rules for $B \rightarrow E_1 \text{ rel } E_2 \text{ Ps}$

$$test = E_1.addr \text{ rel.op } E_2.addr$$

$S = \text{if } B.true \neq \text{fall and } B.false \neq \text{fall then}$

$$\text{gen}('if' \text{ test } 'goto' B.true) \parallel \text{gen}('goto' B.false)$$

$\text{else if } B.true \neq \text{fall then gen}('if' \text{ test } 'goto' B.true)$

$\text{else if } B.false \neq \text{fall then gen}('ifFalse' \text{ test } 'goto' B.false)$

else ''

$$B.code = E_1.code \parallel E_2.code \parallel S$$

If both $B.true$ & $B.false$ are explicit labels i.e. neither is fall.

If $B.true$ is an explicit label, then $B.false$ must be fall, so they create an if instruction that lets control fall through if the condition is false.

If $B.false$ is an explicit label then they generate an ifFalse instruction.

The semantic rule for $B \rightarrow B_1 \parallel B_2 \&$,

$$B_1.true = \text{if } B.true \neq \text{fall then } B.true \text{ else newlabel()}$$

$$B_1.false = \text{fall}$$

$$B_2.false = B.true$$

$$B_2.true = B.false$$

$$B.code = \text{if } B.true \neq \text{fall then } B_1.code \parallel B_2.code$$

$$\text{else } B_1.code \parallel B_2.code \parallel \text{label}(B_1.true)$$

Here note that the meaning of label fall for B is different from its meaning for B₁. Suppose B₁.true is fall i.e control flows falls through B, if B evaluates to true. Although B evaluates to true if B₁ does, B₁.true must ensure that control jumps over the code for B₂ to get to the next instruction after B.

On the other hand, if B₁ evaluates to false, the truth value of B is determined by the value of B₂.

Ex: If $(x < 100 \vee x > 200 \wedge x \neq y)$ $x = 0$;

translates into.

If $x < 100$ goto L₂

If False $x > 200$ goto L₁

If False $x \neq y$ goto L₁

L₂: $x = 0$

L₁:

Boolean values and Jumping code.

A boolean expression may also be evaluated for its value, as in assignment statement such as $x = \text{true}$ or $x = a < b$;

A clean way of handling both roles of boolean expressions is to first build a syntax tree for expression, using either of the following approaches.

(1) use 2 passes. Construct a complete syntax tree for the input & then walk the tree in depth first order, computing

the translations specified by the semantic rules.
(12) Use one pass for statements, but 2 pass for expressions.
With this approach, we would translate E in $\text{while}(E) S_1$ before S_1 is examined.

Consider

$$S \rightarrow \text{Id} = E ; \mid \text{if}(E) S_1 \mid \text{while}(E) S \mid S_1 S_2$$
$$E \rightarrow E_1 E_2 \mid E_1 \text{rel} E_2 \mid E + E \mid (E) \mid \text{true} \mid \text{false}$$

Nonterminal E governs the flow of control in

$S \rightarrow \text{while}(E) S_1$. The same E denotes a value $S \rightarrow \text{Id} = E$ and $E \rightarrow E + E$.

Suppose $E.n$ denotes the syntax tree node for an expression E . Let method `jump` generate jumping code at an expression node & `rvalue` generate code to compute the value of the node into a temporary.

When E appears in $S \rightarrow \text{while}(E) S_1$, method `jump` is called at node $E.n$. Jumping code is generated by calling $E.n.\text{jump}(t, f)$ where t is a new label for S_1 code & f is the label `S.next`.

When E appears in $S \rightarrow \text{Id} = E$; method `rvalue` is called at node $E.n$. If E has the form $E_1 + E_2$ the method call $E.n.\text{rvalue}()$ generates code. If E has the form $E_1 \text{rel} E_2$ we first generate jumping code for E & then assign true or false to a new temporary t at the true & false exits respectively from the jumping code.

Ex: the assignment $x = a < b \ \&\& \ c < d$ can be implemented by the code

If False $a < b$ goto L₁

If False $c < d$ goto L₁

t = true

goto L₂

L₁ : t = false

L₂ : x = t

Backpatching

A key problem when generating code for boolean expressions & flow of control statement is that matching a jump instruction with the target of the jump.

This can be solved by an approach, called backpatching, in which lists of jumps are passed as synthesized attributes. Specifically, when a jump is generated, the target of the jump is temporarily left unspecified. Each such jump is put on a list of jumps whose labels are to be filled in when the proper label can be determined. All the jumps on a list have the same target label.

One-pass code Generation using Backpatching

Backpatching can be used to generate code for boolean expressions & flow of control statements in one pass.

Synthesized attributes trueList & falseList of nonterminal B are used to manage labels in jumping code for boolean expression.

$B.trueList \rightarrow$ list of jump or conditional jump instructions into which we must insert the label to which control goes if B is true.

$B.falseList \rightarrow$ list of instructions that eventually get the label to which control goes when B is false.

As code is generated for B , jumps to the true & false exit are left incomplete, with the label field unfilled.

S has a synthesized attribute $S.nextList$, denoting a list to jump to the instruction immediately following the code for S .

To manipulate lists of jumps, we use 3 functions

(i) $makeList(l)$ creates a new list containing only l , an index into the array of instructions, $makeList$ returns a pointer to the newly created list.

(ii) $merge(p_1, p_2)$ concatenates the lists pointed to by p_1 & p_2 & returns a pointer to the concatenated list.

(iii) $backpatch(p, l)$ inserts l as the target label for each of the instructions on the list pointed to by p .

Backpatching for Boolean Expressions

We can construct a translation scheme suitable for generating code for boolean expression during bottom-up parsing. A marker nonterminal M in the grammar causes a semantic action to pick up, at appropriate time, the index of the next instruction to be generated. The grammar is,

$$B \rightarrow B_1 \parallel M B_2 \mid B_1 \& \& M B_2 \mid ! B_1 \mid (B_1) \mid E_1 \text{ rel } E_2 \mid \text{true} \mid \text{false}$$

$M \rightarrow E$

The translation scheme is,

- (i) $B \rightarrow B_1 \parallel M B_2$ { backpatch (B_1 .false_{list}, M .instr);
 B .true_{list} = merge (B_1 .true_{list}, B_2 .true_{list});
 B .false_{list} = B_2 .false_{list}; }
- (ii) $B \rightarrow B_1 \&\& M B_2$ { backpatch (B_1 .true_{list}, M .instr);
 B .true_{list} = B_2 .true_{list};
 B .false_{list} = merge (B_1 .false_{list}, B_2 .false_{list}); }
- (iii) $B \rightarrow ! B_1$ { B .true_{list} = B_1 .false_{list};
 B .false_{list} = B_1 .true_{list}; }
- (iv) $B \rightarrow (B_1)$ { B .true_{list} = B_1 .true_{list};
 B .false_{list} = B_1 .false_{list}; }
- (v) $B \rightarrow E_1 \text{ rel } E_2$ { B .true_{list} = make_{list} (next instr);
 B .false_{list} = make_{list} (next instr + 1);
 gen ('if' E_1 .addr rel.op E_2 .addr 'goto -');
 gen ('goto -'); }
- (vi) $B \rightarrow \text{true}$ { B .true_{list} = make_{list} (next instr);
 gen ('goto -'); }
- (vii) $B \rightarrow \text{false}$ { B .false_{list} = make_{list} (next instr);
 gen ('goto -'); }
- (viii) $M \rightarrow E$ { M .instr = next instr; }

Consider (c) $B \rightarrow B_1 \parallel M B_2$. If B_1 is true, then B is also true, so B_1 .trueList becomes part of B .trueList. If B_1 is false, test B_2 , so the target for the jumps B_1 .falseList must be beginning of the code generated for B_2 . This target is obtained by M . It produces synthesized attribute M .instr, the index of the next instruction, just before B_2 code starts being generated. To obtain that instruction index, we associate with the production $M \rightarrow e$

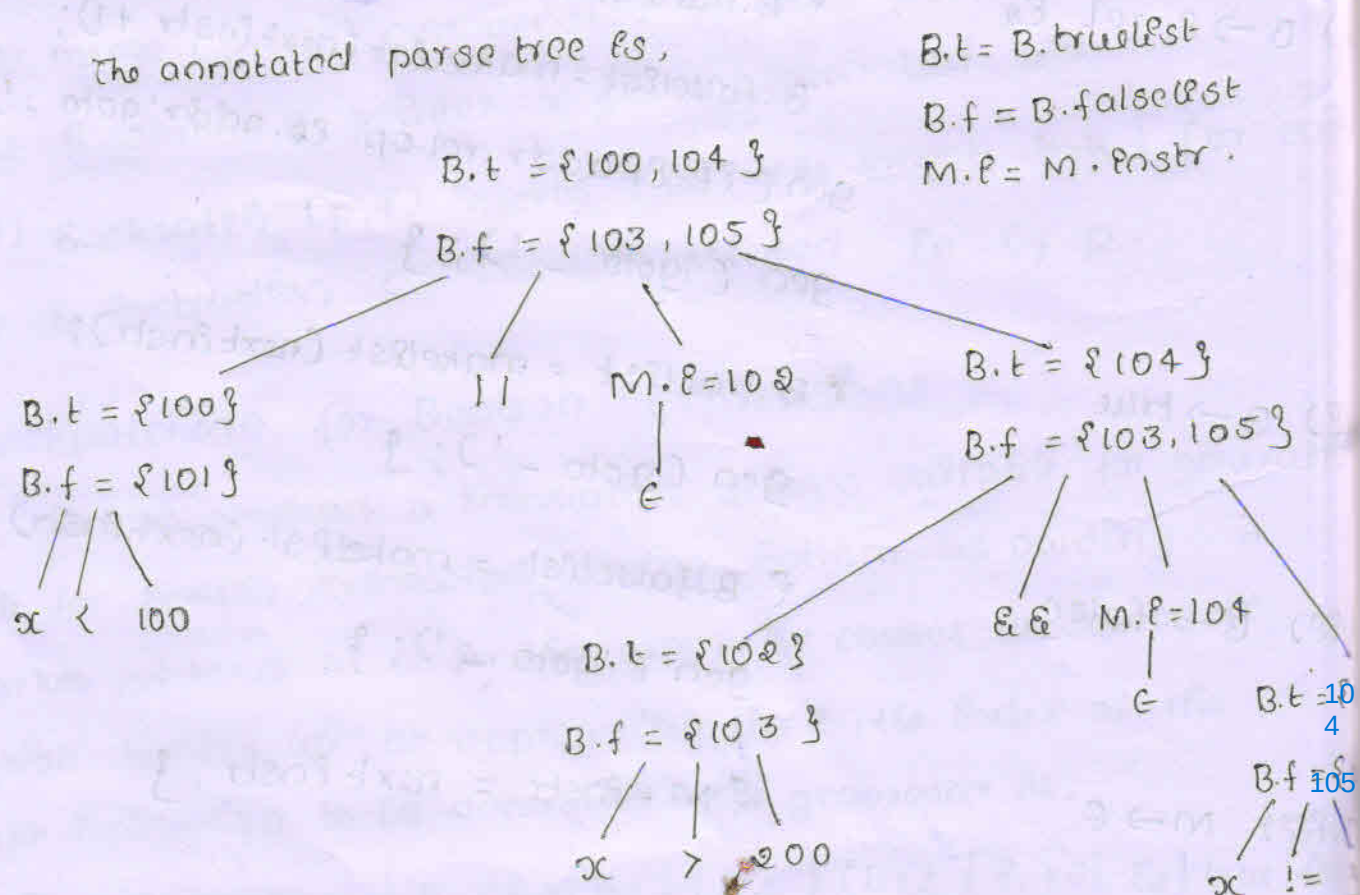
$\{ M.instr = nextInstr; \}$

The variable $nextInstr$ holds the index of the next instruction to follow. This value will be backpatched onto B_1 .falseList when we have seen the remainder of the production $B \rightarrow B_1 \parallel M$.

Example:

Consider the expression $x < 100 \parallel x > 200 \ \&\& \ x \neq y$

The annotated parse tree is,



The reduction of $x < 100$ to B by production (v), the
2 instructions are generated

100: If $x < 100$ goto —

101: goto —

The marker nonterminal M in the production

$B \rightarrow B_1 \mid MB_2$

records the value of nextinstr, which at this time is 102.
The reduction of $x > 200$ to B by production (v) generate
the instructions

102: If $x > 200$ goto —

103: goto —

The sub expression $x > 200$ correspond to B_1 in the
production $B \rightarrow B_1 \&\& MB_2$

The marker nonterminal M records the current value of
nextinstr, which is now 104. Reducing $x \neq y$ into B by
production (v) generates

104: If $x \neq y$ goto —

105: goto —

We now reduce by $B \rightarrow B_1 \&\& MB_2$, the corresponding
semantic action calls backpatch (B_1 .trueinst, M.instr) to bind
the true exit of B_1 to the first instruction of B_2 . Since
 B_1 .trueinst is 102 & M.instr is 104 this call to backpatch
fills in 104 in instruction 102. The six instructions generated
so far are

100: If $x < 100$ goto —

101: goto —

102: if $x > 200$ goto 104

103: goto —

104: if $x \neq y$ goto —

105: goto —

The semantic action associated with the final production by $B \rightarrow B_1 \mid M B_2$ calls backpatch ($\{101\}, 102$) which leaves the instruction as

100: if $x < 100$ goto —

101: goto 102

102: if $x > 200$ goto 104

103: goto —

104: if $x \neq y$ goto —

105: goto —

The entire expression is true iff gotos of instructions 100 or 104 are reached, & is false iff gotos of instructions 103 or 105 are reached.

Flow of control statements

We can use backpatching to translate flow of control statements in one pass. Consider

$S \rightarrow \text{if } (B) S \mid \text{if } (B) S \text{ else } S \mid \text{while } (B) S \mid \text{do } \{ A \}$

$L \rightarrow LS \mid S$

where $S \rightarrow$ statement $L \rightarrow$ statement list

$A \rightarrow$ assignment statement $B \rightarrow$ Boolean expression

The translation scheme is,

(e) $S \rightarrow \text{if } (B) M S_1$ { backpatch (B.trueList, M.instr);
S.nextList = merge (B.falseList, S₁.nextList); }

(ee) $S \rightarrow \text{if } (B) M S_1 N \text{ else } M_2 S_2$
{ backpatch (B.trueList, M₁.instr);
backpatch (B.falseList, M₂.instr);
temp = merge (S₁.nextList, N.nextList);
S.nextList = merge (temp, S₂.nextList); }

(eee) $S \rightarrow \text{while } M_1 (B) M_2 S_1$
{ backpatch (S₁.nextList, M₁.instr);
backpatch (B.trueList, M₂.instr);
S.nextList = B.falseList;
gen ('goto' M.instr); }

(ev) $S \rightarrow \{ L \}$ { S.nextList = L.nextList; }

(v) $S \rightarrow A;$ { S.nextList = null; }

(ve) $M \rightarrow E$ { M.instr = nextInstr; }

(vve) $N \rightarrow E$ { N.nextList = makeList (nextInstr);
gen ('goto -'); }

(vvee) $L \rightarrow L_1 M S$ { backpatch (L₁.nextList, M.instr);
L.nextList = S.nextList; }

(vxx) $L \rightarrow S$ { L.nextList = S.nextList; }

Consider the production $S \rightarrow \text{while } M_1(B) M_2 S_1$. The occurrences of marker nonterminal M in the production record the instruction numbers of the beginning of the code for B the beginning of the code for S_1 . The production $M \rightarrow \epsilon$ sets attribute $M.\text{instr}$ to the no. of next instruction. After S_1 is executed, control flows to the beginning. Therefore, when reduce to S , we backpatch $S_1.\text{nextlist}$ to make all targets on that list be $M_1.\text{instr}$. An explicit jump to the beginning of the code for B is appended after the code for S_1 because control may also "fall out the bottom". $B.\text{truelist}$ is backpatched to go to the beginning of S_1 by making jumps on $B.\text{true}$ go to $M_2.\text{instr}$.

$S.\text{nextlist}$ & $L.\text{nextlist}$ comes when code is generated for $\text{if}(B) S_1 \text{ else } S_2$. If control "falls out the bottom" of S_1 as when S_1 is an assignment, we must include at the end of the code for S_1 a jump over the code for S_2 . We use another marker nonterminal to generate this jump after S_1 , i.e. N .

We backpatch the jumps when B is true to $M_1.\text{instr}$ the latter is the beginning of the code for S_1 . Similarly we backpatch jumps when B is false to go to the beginning of the code for S_2 . The list $S.\text{nextlist}$ includes all jumps out of S_1 as well as the jump generated by N .

Semantic actions (val) & (ex) handles sequence of statements

$L \rightarrow L_1 M S$

In the instruction following the code for L_1 in order of execution is the beginning of S . Thus the $L_1.\text{nextlist}$ list is backpatched to the beginning of the code for S , which is given by $M.\text{instr}$.

On $L \rightarrow S$, $L.nextlist$ is same as $S.nextlist$.

Break-, Continue - & Goto-Statements.

Goto statements can be implemented by maintaining a list of unfilled jumps for each label and then backpatching the target when it is known.

The break statements sends control out of an enclosing loop & continue statement triggers the next iteration of an enclosing loop.

Consider

```
(e) for ( ; ; readch ( ) ) {  
(ee) if ( peek == ' ' || peek == '\t' ) continue;  
(eee) else if ( peek == '\n' ) line = line + 1;  
(ev) else break;  
(v) }
```

ee. Break statement on line (ev) make control to jump to the next statement after the enclosing for loop.
Continue on line (ee) makes control to jump to readch()

If S is the enclosing loop construct, then a break statement is a jump first instruction after the code for S . We can generate code for the break

(e) keeping track of the enclosing loop statement S

(ee) Generating an unfilled jump for the break statement

(eee) Putting this unfilled jump on $S.nextlist$

In 2 pass front end S_{next} can be implemented as a field in the node for S . We can keep track of S by using the symbol table to map a special identifier back to the for the enclosing loop statement S .

Instead of using the symbol table we can put pointer to S_{next} in the symbol table. When a break statement is reached, we generate an unfilled jump, look up $next$ through the symbol table & add the jump to the list, where it will be backpatched.

Continue statement can be handled in a manner analogous to the break statement. The main difference b/w the 2 is the target of the generated jump is different.

* Switch - Statements

The switch statement syntax is,

switch (E) {

case V_1 : S_1

case V_2 : S_2

...

case V_{n-1} : S_{n-1}

default : S_n

}

There is a selector expression E , which is to be evaluated followed by n constant values $V_1, V_2, \dots, V_{n-1}$ that the expression might take, including a default "value" which always matches the expression if no other value does.

Translation of Switch - statements.

The intended translation of a switch is code to

- (i) Evaluate the expression E
- (ii) Find the value v_j in the list of cases that is the same as the value of the expression.
- (iii) execute the statement S_j associated with the value found.

Step (i) is an n -way branch, if the no. of cases is small then it is reasonable to use a sequence of conditional jump, each tests for an individual value & transfer to the code

A compact way to implement this is to create a table of pairs, each consist of value & label for statement's code. The value of the expression itself, paired with label for the default statement. If no match found, then last entry is sure to match.

If no. of values is more, it is more efficient to construct a hash table, with the labels of various statements as entries. If no entry for value possessed by the switch is found, a jump to the default statement is generated.

There is a common special case that can be implemented even more efficiently than by an n -way branch. If the values all lie in some small range say $\min$ to $\max$ & no. of diff. values is a reasonable fraction of $\max - \min$, then we can construct an array of $\max - \min$ "buckets", where bucket $j - \min$ contains the label of the statement with value j ; any bucket that would otherwise remain unfilled contains the default label.

To perform switch, evaluate the expression to obtain j & check that j is in the range $\min$ to $\max$ & transfer indirectly to the table entry at offset $j - \min$.

Syntax-Directed Translation of switch-statement.

The translation of a switch statement is,

```
      goto test
L1 : code for S1
      goto next
L2 : code for S2
      goto next
      ...
Ln-1 : code for Sn-1
      goto next
Ln : code for Sn
      goto next
test : if t = v1 goto L1
      if t = v2 goto L2
      ...
      if t = vn-1 goto Ln-1
      goto Ln
```

Next:

When we see the keyword switch, we generate a new label test & next, and a new temporary t . Then, as we parse the expression E , we generate code to evaluate E into t . After processing E , we generate the jump goto test.

When we see each case keyword, we create a new label L_i & enter it into the symbol table. We place in a queue, used only to store cases, a value-label pair consisting of the value v_i of the case constant & L_i . We process each statement case v_i : S_i by emitting the label L_i attached to the code for S_i followed by the jump goto next.

When the end of the switch is found, we are ready to generate the code for the n-way branch. Reading the queue of value-label pairs, we can generate a sequence of 3-address statements of the form.

case $t = v_1$ L_1

case $t = v_2$ L_2

case $t = v_{n-1}$ L_{n-1}

case $t = t$ L_n

next;

where t is the temporary holding the value of the selector expression & L_n is the label for the default statement.

The case $t = v_i$ L_i instruction is a synonym for
if $t = v_i$ goto L_i .

Intermediate code for Procedures.

In 3-address code, a function call is unraveled into the evaluation of parameters in preparation for a call, followed by the call itself.

Ex: a is an array of integer & that f is a function from integers to integers. then the assignment

$$n = f(a[i]);$$

might translate into the following 3-address code

$$(l) \quad t_1 = i * 4$$

$$(ll) \quad t_2 = a[t_1]$$

$$(lll) \quad \text{param } t_2$$

$$(lv) \quad t_3 = \text{call } f, 1$$

$$(v) \quad n = t_3$$

The first 2 lines compute the value of expression $a[i]$ into t_2 , Line (lll) makes t_2 an actual parameter for the call of f on line (lv). That line also assigns the return value to temporary t_3 . Line (v) assigns the result of $f(a[i])$ to n .

The below productions allow function definitions & function calls.

$$D \rightarrow \text{define } T \text{ Id } (F) \{S\}$$

$$F \rightarrow \epsilon \mid T \text{ Id}, F$$

$$S \rightarrow \text{return } E;$$

$$E \rightarrow \text{Id}(A)$$

$$A \rightarrow \epsilon \mid E, A$$

* Non terminal D & T general declaration and type

* D consists of keyword define, a return type, the function name, formal parameters in parentheses & function body

consisting of a bracketed statement.

- * F generates zero or more formal parameters

- * S & E generate statements & expressions.

- * S adds a statement that returns the value of an expression

- * E adds function call with actual parameter generated by A

- * Actual parameter is an expression.

Function types: the type of a function must encode the return type and types of the formal parameters.

Symbol tables: Let s be the top symbol table when the function definition is reached. In the production for D, after seeing `define` & the function name, we push s & set up a new symbol table.

`Env.push(top); top = new Env(top);`

Type checking: Within expressions, a function is treated like any other operator. For ex, if f is a function with a parameter of type `real`, then the integer 2 is coerced to a `real` in the call $f(2)$.

Function calls: When generating 3-address instructions for a function call $f(e_1, e_2, \dots, e_n)$ it is sufficient to generate the 3-address instructions for evaluating or reducing the parameters e_i to addresses, followed by a param instruction for each parameter.

QUESTIONS

1. Define quadruples, triples and static single assignment form.

A quadruple has 4 fields, op, arg1, arg2 & result. The op field contains an incremental code for the operation.

Ex: the quadruples for $a = (6 * -c) + (6 * -c)$

	op	arg1	arg2	result
$t_1 = \text{minus } c$	0	minus	c	t_1
$t_2 = 6 * t_1$	1	*	6, t_1	t_2
$t_3 = \text{minus } c$	2	minus	c	t_3
$t_4 = 6 * t_3$	3	*	6, t_3	t_4
$t_5 = t_2 + t_4$	4	+	t_2 , t_4	t_5
$a = t_5$	5	=	t_5	a

A triple has only 3 fields op, arg1 & arg2.

Ex: the triples for $a = 6 * -c + 6 * -c$.

	op	arg1	arg2
0	-minus	c	
1	*	6	(0)
2	minus	c	
3	*	6	(2)
4	+	(1)	(3)
5	=	a	(4)

Static single-assignment form is an intermediate representation that facilitates certain code optimizations

Ex:

$$p = a + b$$

$$q = p - c$$

$$p = q * d$$

$$p = e - p$$

$$q = p + q$$

(a) 3-address code

$$p_1 = a + b$$

$$q_1 = p_1 - c$$

$$p_2 = q_1 * d$$

$$p_3 = e - p_2$$

$$q_2 = p_3 + q_1$$

(b) Static single assignment form.

Q. Develop SDD to produce directed acyclic graph for an expression show the steps for constructing the DAG for the expression $a + a * (b - c) + (b - c) * d$.

Syntax directed definition is,

PRODUCTION	SEMANTIC RULES
(E) $E \rightarrow E_1 + T$	$E.\text{node} = \text{new Node}('+', E_1.\text{node}, T.\text{node})$
(EE) $E \rightarrow E_1 - T$	$E.\text{node} = \text{new Node}('-', E_1.\text{node}, T.\text{node})$
(EEE) $E \rightarrow T$	$E.\text{node} = T.\text{node}$
(Ev) $T \rightarrow (E)$	$T.\text{node} = E.\text{node}$
(V) $T \rightarrow id$	$T.\text{node} = \text{new Leaf}(id, id.\text{entry})$
(vi) $T \rightarrow \text{num}$	$T.\text{node} = \text{new Leaf}(\text{num}, \text{num.val})$

Steps for constructing the DAG

(e) $P_1 = \text{Leaf}(\text{Id}, \text{entry-a})$

(ee) $P_2 = \text{Leaf}(\text{Id}, \text{entry-a}) = P_1$

(eee) $P_3 = \text{Leaf}(\text{Id}, \text{entry-b})$

(ev) $P_4 = \text{Leaf}(\text{Id}, \text{entry-c})$

(v) $P_5 = \text{Node}('-', P_3, P_4)$

(ve) $P_6 = \text{Node}('*', P_1, P_5)$

(vlee) $P_7 = \text{Node}('+', P_1, P_6)$

(~~ex~~) (vlee) $P_8 = \text{Leaf}(\text{Id}, \text{entry-b}) = P_3$

(ex) $P_9 = \text{Leaf}(\text{Id}, \text{entry-c}) = P_4$

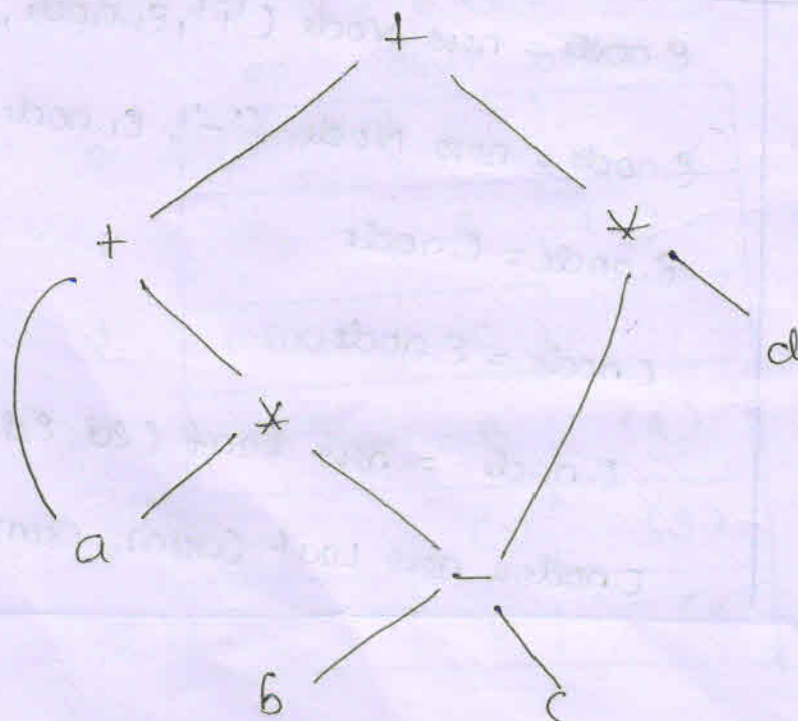
(x) $P_{10} = \text{Node}('-', P_3, P_4) = P_5$

(xe) $P_{11} = \text{Leaf}(\text{Id}, \text{entry-d})$

(xee) $P_{12} = \text{Node}('*', P_5, P_{11})$

(xlee) $P_{13} = \text{Node}('+', P_7, P_{12})$

DAG



3. Write syntax directed definition for flow of control statement

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = nextlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow assign$	$S.code = assign.code$
$S \rightarrow if(B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow if(B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow while(B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$

	$S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next)$ $\parallel S_2.code.$
--	---

4. Explain syntax directed translation of switch statement.

Translation of a switch statement

goto test

L_1 : code for S_1

goto next

L_2 : code for S_2

goto next.

...

L_{n-1} : code for S_{n-1}

goto next

L_n : code for S_n .

goto next

test : if $t = v_1$ goto L_1

if $t = v_2$ goto L_2

...

if $t = v_{n-1}$ goto L_{n-1}

goto L_n

next :

Fig(1)

We can write more straightforward sequence as,

if $t = v_1$ goto L_1

code for S_1

goto next

L_1 : if $t \neq v_2$ goto L_2

```

        code for S2
        goto next

L1 : if t != v2 goto L2
        code for S2
        goto next

L2 :
        . . .

Ln-2 : if t != vn-1 goto Ln-1
        code for Sn-1
        goto next

Ln-1 : code for Sn

next :

```

Above code require the compiler to do extensive analyses to find the most efficient implementation. It is inconvenient in a one pass compiler.

To translate into the form of fig (1), when we see the keyword switch, we generate a new labels test & next, & a new temporary t. Then, as we parse the expression E, we generate code to evaluate E to t. After processing E, we generate the jump goto test. When we see each case keyword, we create a new table LP & enter it into the symbol table. We place in a queue the value VP of the case constant LP. We process each statement case VP: SP by emitting the label LP attached to the code for SP, followed by the jump goto next.

When the end of the switch is found, we are ready to

generate the code for the n way branch. code is

case $t \vee L_1$

case $t \vee L_2$

...

case $t \vee L_{n-1}$

case $t \vee L_n$

next :

where, $t \rightarrow$ temporary holding the value of the selector expression

$L_n \rightarrow$ label for the default statement.

case $t \vee L_P$ is same as if $t = v$ goto L_P .